

Jamaica A.I. Explorer

Architecture, Framework and Hosting Overview

Field	Detail
Repository	github.com/J-A-I-A/jamaica-ai-explorer
Branch documented	main
Application name	National A.I. Policy Explorer — Jamaica
Package identifier	jaia v0.1.0 (private)
Primary framework	Next.js 16.2.11 (App Router) on React 19.2.4
Language	TypeScript 5 (strict mode)
Source size	~2,485 lines of TypeScript/TSX across 22 source files
Document date	27 July 2026 (revised 11 August 2026 — feedback storage)

1. Purpose of the Application

The Jamaica A.I. Explorer is an interactive web presentation of the report National Artificial Intelligence — Policy Recommendations, prepared by Jamaica's National Artificial Intelligence Task Force and presented to the Office of the Prime Minister. It converts a static PDF report into a navigable, searchable and filterable web experience, and layers a document-grounded A.I. assistant and a public feedback channel on top of it.

The application is content-centric rather than transactional. There is no database or content management system for the policy content itself, and no authentication layer: the entire body of policy content is held in typed TypeScript modules compiled into the build. The one piece of persistent state is public feedback, which is written to a Postgres database (section 6.2). The two dynamic capabilities — the assistant and the feedback form — are the only server-side workloads.

2. Framework and Technology Stack

The project is a single-repository, full-stack Next.js application. Front-end rendering and back-end API endpoints are served by the same framework and deployed as one unit.

2.1 Runtime and framework dependencies

Package	Version	Role in the architecture
next	16.2.11	App Router framework — routing, React Server Components, static generation, API route handlers, image and font optimisation

Package	Version	Role in the architecture
<code>react / react-dom</code>	<code>19.2.4</code>	UI runtime; Server Components by default with selective client hydration
<code>openai</code>	<code>^6.49.0</code>	Chat completions client for the assistant; pointed at any OpenAI-compatible endpoint
<code>pg</code>	<code>^8.13.0</code>	Postgres client (node-postgres) used by <code>src/lib/db.ts</code> to persist feedback submissions
<code>react-markdown</code>	<code>^10.1.0</code>	Renders streamed Markdown responses from the assistant
<code>remark-gfm</code>	<code>^4.0.1</code>	GitHub-flavoured Markdown extensions (tables, strikethrough, task lists)
<code>@vercel/analytics</code>	<code>^2.0.1</code>	Client-side page-view and traffic analytics, mounted in the root layout

2.2 Build and development toolchain

Package	Version	Role in the architecture
<code>typescript</code>	<code>^5</code>	Strict type checking; path alias <code>@/*</code> mapped to <code>./src/*</code>
<code>tailwindcss</code>	<code>^4</code>	Utility-first styling, configured CSS-first via <code>@theme inline</code>
<code>@tailwindcss/postcss</code>	<code>^4</code>	PostCSS plugin that compiles the Tailwind v4 pipeline
<code>eslint</code>	<code>^9</code>	Flat-config linting
<code>eslint-config-next</code>	<code>16.2.11</code>	Core Web Vitals and TypeScript rule sets
Yarn	(<code>nodeLinker: node-modules</code>)	Package manager; lockfile committed as <code>yarn.lock</code>

Note: `package.json` exposes both a Webpack development script (`next dev --webpack`, with `WATCHPACK_POLLING` set for containerised or network file systems) and a Turbopack alternative (`dev:turbo`). `next.config.ts` pins the Turbopack root to the repository directory, which prevents workspace root mis-detection when the project sits alongside other package manifests.

3. Repository Structure

The repository is flat and conventional for a Next.js App Router project. All application code lives under `src/`, with a three-way separation between routes (`src/app`), presentational components (`src/components`) and content (`src/data`).

```
jamaica-ai-explorer/
├── next.config.ts      Next.js configuration (Turbopack root)
├── tsconfig.json       Strict TS, bundler resolution, @/* → ./src/*
├── postcss.config.mjs  Tailwind v4 PostCSS pipeline
├── eslint.config.mjs   Flat ESLint config (Next core-web-vitals + TS)
├── package.json / yarn.lock  Dependency manifest and lockfile
├── public/            Static assets: coat-of-arms.svg, icons, robots.txt
├── src/
│   ├── app/           App Router – routes, layouts, API handlers
│   │   ├── layout.tsx  Root layout: fonts, theme bootstrap, chrome
│   │   ├── globals.css Design tokens and Tailwind theme mapping
│   │   ├── page.tsx    Landing page
│   │   ├── explore/    Explorer index + [slug] pillar detail pages
│   │   ├── roadmap/ swot/ ethics/ task-force/ Content routes
│   │   ├── assistant/ feedback/ Interactive routes
│   │   └── api/
│   │       ├── chat/route.ts Streaming assistant endpoint
│   │       └── feedback/route.ts Feedback intake endpoint
│   ├── components/    8 client components + shared UI primitives
│   ├── lib/           Server-only: db.ts – Postgres pool, schema, feedback inserts
│   └── data/          recommendations.ts, context.ts, documentContext.ts
```

4. Application Architecture

4.1 Rendering model

The application uses the Next.js App Router with React Server Components as the default. Every route module is a server component that imports content directly from the data layer and renders it to HTML at build time. Interactivity is isolated to eight components carrying the "use client" directive, which are the only modules shipped to the browser as hydrated JavaScript.

This produces a hybrid deployment profile:

- **Static (prerendered at build):** all content routes. The nine pillar detail pages are enumerated ahead of time by `generateStaticParams()` in `src/app/explore/[slug]/page.tsx`, so no pillar page is rendered on demand.
- **Dynamic (server-executed per request):** only the two API route handlers, both of which declare `export const dynamic = "force-dynamic"` and `export const runtime = "nodejs"`.

The practical consequence is that the site behaves as a static site for all reading traffic, with two serverless-style functions handling the interactive features.

4.2 Route map

Route	Rendering	Responsibility
/	Static	Hero, vision statement, nine policy pillars, horizon counts, global themes

Route	Rendering	Responsibility
<code>/explore</code>	Static shell + client state	Full-text search with match highlighting; multi-select horizon and pillar filters; grouped or flat views
<code>/explore/[slug]</code>	Static (9 pages)	Per-pillar detail: policy issue, challenges, recommendations grouped by horizon, previous/next navigation
<code>/roadmap</code>	Static shell + client state	Recommended action plan as a pillar × horizon matrix; click a horizon to isolate it
<code>/swot</code>	Static shell + client state	SWOT analysis; click a quadrant to expand it full-width
<code>/ethics</code>	Static	Ethical considerations, comparative global themes, references
<code>/task-force</code>	Static	Chair, the sixteen members, and the mandate
<code>/assistant</code>	Static shell + client state	Chat interface for the document-grounded assistant
<code>/feedback</code>	Static shell + client state	Public feedback form with reCAPTCHA
<code>/api/chat</code>	Dynamic (Node)	Streaming chat completions proxy
<code>/api/feedback</code>	Dynamic (Node)	Validation, bot verification and delivery of feedback submissions

4.3 Server / client component split

Eight components are marked as client components. Everything else — including all page modules and the entire data layer — remains server-side and never reaches the browser bundle.

Client component	Reason for client execution
<code>Explorer.tsx</code>	Search input state, filter selection, match highlighting (261 lines)
<code>Assistant.tsx</code>	Chat state, streaming fetch reader, auto-scroll (215 lines)
<code>FeedbackForm.tsx</code>	Controlled form fields, reCAPTCHA widget lifecycle, submission state (304 lines)
<code>Roadmap.tsx</code>	Horizon isolation on click within the pillar × horizon matrix
<code>SwotBoard.tsx</code>	Quadrant expand/collapse interaction
<code>SiteHeader.tsx</code>	Navigation state and active-route highlighting
<code>ThemeToggle.tsx</code>	Reads and writes the theme preference in localStorage

Client component	Reason for client execution
<code>Markdown.tsx</code>	Renders streamed assistant Markdown with GFM support

5. Content and Data Layer

There is no database and no external content fetching. The complete report is modelled as typed TypeScript constants that the compiler validates and the bundler inlines.

Module	Contents
<code>src/data/recommendations.ts</code>	The nine policy pillars — each with slug, icon, policy issue, objective, challenges and recommended actions tagged short medium long. Also defines the HORIZONS map (1–3, 4–6 and 7–10 year bands) and the national vision statement. 383 lines.
<code>src/data/context.ts</code>	SWOT analysis, ethical considerations, global thematic areas, task force chair and membership roster, and references. 202 lines.
<code>src/data/documentContext.ts</code>	Server-only module (imports "server-only") that flattens the two modules above into a compact structured knowledge base string, and exports the assistant system prompt. 85 lines.

Two design consequences follow from this model:

- **Derived exports guarantee consistency.** `ALL_ACTIONS` and `COUNTS` are computed from the pillar definitions rather than maintained by hand, so the statistics on the landing page and the result counts in the explorer cannot drift from the underlying content.
- **Content edits are code edits.** Amending policy text means editing the two data modules and redeploying; every page, filter and count updates from that single source. This is deliberate — it removes an entire class of CMS infrastructure, at the cost of requiring a build to publish a correction.

6. API Layer

6.1 /api/chat — document-grounded assistant

A streaming proxy between the browser and any OpenAI-compatible chat completions endpoint. It deliberately does not use a vector store or retrieval pipeline; instead the entire report is compressed into a structured system prompt built by `buildDocumentContext()`, which keeps the assistant grounded without any retrieval infrastructure.

- Provider-agnostic by configuration. The OpenAI SDK is pointed at `OPENAI_BASE_URL`, so the same code path serves `api.openai.com`, an Azure OpenAI `".../openai/v1"` deployment, or a local server such as Ollama at `http://localhost:11434/v1`. A keyless local endpoint is supported.
- Input hardening. Messages are sanitised before they leave the server: role values are restricted to user | assistant, each message is trimmed to 6,000 characters, history is capped at the last 20 messages, and the window is shifted so it always begins with a user turn.
- Streaming response. The upstream completion stream is re-encoded into a `ReadableStream` of UTF-8 text delivered as `text/plain` with `Cache-Control: no-store`. Client disconnects abort the upstream request via the completion controller.

- Graceful degradation. With no API key and no base URL configured, the endpoint returns HTTP 503 with an explanatory message rather than failing; upstream errors return 502; mid-stream failures append an inline error notice instead of severing the connection.
- Generation parameters. Temperature 0.3 and a default output ceiling of 4,096 tokens (MODEL_MAX_TOKENS), sized to accommodate reasoning models that consume much of their budget on hidden reasoning.

6.2 /api/feedback — public submission intake

A validate-verify-deliver pipeline for public comment on the policy recommendations.

- Validation. Field length limits are enforced server-side (name 120, email 160, organisation 160, message 5,000 characters); the topic is checked against a fixed allow-list of eleven values and falls back to "General"; email format is regex-validated when supplied.
- Bot protection. Google reCAPTCHA v2 is verified server-side against the siteverify endpoint, with the client IP forwarded from the x-forwarded-for header. Verification is enforced only when RECAPTCHA_SECRET_KEY is present, so local development runs without it.
- Durable delivery. Each submission is assigned a UUID and ISO timestamp, then written to a Postgres feedback_entries table by src/lib/db.ts. All entries belonging to one submission are inserted in a single transaction — the whole submission is stored or none of it is — and inserts are idempotent (ON CONFLICT DO NOTHING on the entry id), so a client retry cannot create duplicates. The table is created on first use (CREATE TABLE IF NOT EXISTS) rather than by a separate migration step. Connection details come from POSTGRES_URL, with optional POSTGRES_USER/POSTGRES_PASSWORD overrides and TLS handling that defaults by host and can be overridden with POSTGRES_SSL.
- Best-effort local backup. In addition to the Postgres record, a copy is appended to .data/feedback.jsonl on the server filesystem. This write is wrapped so it never blocks a submission — see the hosting caveat in section 8.3.

7. Design System and Theming

Styling uses Tailwind CSS v4 in its CSS-first configuration: there is no `tailwind.config.js`. Design tokens are declared as CSS custom properties in `src/app/globals.css` and exposed to Tailwind through an `@theme` inline block, which generates utilities such as `bg-jm-ink` and `text-jm-gold` directly from the token names.

Token	Dark (default)	Light	Role
<code>--jm-gold</code>	<code>#ffb400</code>	<code>#8a6100</code>	Primary accent — national gold
<code>--jm-green</code>	<code>#009b3a</code>	<code>#007a2e</code>	Secondary accent — national green
<code>--jm-black</code> / <code>--jm-ink</code>	<code>#0b0b0c</code> / <code>#131417</code>	<code>#f4f2ec</code> / <code>#ffffff</code>	Page ground and surface
<code>--jm-panel</code> / <code>--jm-line</code>	<code>#1a1c20</code> / <code>#2b2e34</code>	<code>#eae7df</code> / <code>#e0ddd3</code>	Card fill and border
<code>--jm-text</code> / <code>--jm-muted</code>	<code>#f2f1ee</code> / <code>#a3a6ad</code>	<code>#16181c</code> / <code>#5c6068</code>	Primary and secondary type

Both themes are defined; a dark ground with gold as the primary accent is the default, and the light palette darkens the gold and green to preserve contrast on a pale ground.

Typography is loaded through `next/font/google` with Latin subsetting and self-hosting: Space Grotesk for display and Inter for body text, each bound to a CSS variable (`--font-display-var`, `--font-sans-var`) that the Tailwind theme maps to `font-display` and `font-sans`.

Theme flash is prevented by an inline script injected into `<head>` in the root layout. It runs before first paint, reads the stored preference from `localStorage`, falls back to the `prefers-color-scheme` media query and then to dark, and sets `data-theme` on the document element. Both `<html>` and `<body>` carry `suppressHydrationWarning` because that attribute is written before React hydrates.

8. Hosting and Deployment

8.1 Source of truth

The canonical source is the public GitHub repository `github.com/J-A-I-A/jamaica-ai-explorer` under the Jamaica Artificial Intelligence Association organisation, with `main` as the default and deployment branch.

8.2 Runtime hosting

The repository contains no committed deployment manifest — there is no `vercel.json`, `Dockerfile`, `netlify.toml`, or `.github/workflows` directory. Hosting is therefore configured outside the repository, and the platform is identified from the runtime instrumentation rather than from configuration.

The evidence points to Vercel:

- `@vercel/analytics ^2.0.1` is a production dependency, and the `<Analytics />` component is mounted in the root layout — this package reports to Vercel's analytics collector and is normally paired with a Vercel deployment.

- `public/vercel.svg` is retained among the static assets.
- The scripts are the unmodified Next.js defaults (next build, next start), which is exactly what Vercel's zero-configuration Next.js pipeline expects.
- The most recent commit on main is titled "added analytics", indicating the analytics integration is a recent deployment-time addition.

On that platform the build output splits as expected: prerendered HTML and static assets are served from the edge CDN, while `/api/chat` and `/api/feedback` are provisioned as Node.js serverless functions.

8.3 Hosting constraints imposed by the code

Constraint	Implication
Node.js runtime is mandatory	Both API routes pin export const runtime = "nodejs" — the OpenAI SDK, the pg driver and the fs write are not Edge-runtime compatible. Neither route can run on an Edge or Workers runtime without modification.
Static-only hosting is not viable	Because of the two dynamic routes, the application cannot be published as a pure static export to GitHub Pages, S3 or a plain object store. It requires either a Node server or a serverless function platform.
Local file writes are ephemeral	The <code>.data/feedback.jsonl</code> backup in the feedback route writes to the function's temporary filesystem. On any serverless platform this file does not persist between invocations and cannot be retrieved. Postgres is the durable sink; the local write should be treated as a development convenience, not a backup of record.
No platform-locked APIs	Aside from analytics, the code uses only standard Next.js and Web APIs. The application is portable to Azure App Service or Container Apps, AWS Amplify or an EC2 Node process, or any container host, with no code changes beyond removing or replacing the analytics component.

8.4 Publication status

The site is currently configured as unpublished and deliberately excluded from indexing. Two independent controls enforce this:

- `public/robots.txt` disallows all user agents across the entire site, with an explicit comment covering search engines and A.I. training crawlers.
- The root layout exports `robots: { index: false, follow: false, nocache: true }`, emitting per-page meta directives, annotated in source as keeping the site out of search results while it is unpublished.

Both controls should be lifted deliberately at launch; leaving either in place will keep the site invisible to search even once it is publicly reachable.

8.5 Build and run

```
# Development
yarn install
yarn dev           # Webpack, polling watcher — http://localhost:3000
yarn dev:turbo     # Turbopack alternative
```

```
# Production
yarn build      # next build – prerenders all static routes
yarn start      # next start – serves static output + API routes

# Quality
yarn lint       # ESLint flat config
```

8.6 Environment variables

No environment file is committed. The following variables must be configured in the hosting platform.

Variable	Scope	Purpose
MODEL_API_KEY or OPENAI_API_KEY	Server	Credential for the chat completions provider. Optional only when a keyless local endpoint is used.
OPENAI_BASE_URL	Server	Provider endpoint. Defaults to OpenAI; set for Azure OpenAI or a self-hosted server.
MODEL or OPENAI_MODEL	Server	Model or deployment name. Defaults to gpt-4o-mini.
MODEL_MAX_TOKENS	Server	Output token ceiling. Defaults to 4096.
RECAPTCHA_SECRET_KEY	Server	Enables server-side reCAPTCHA verification. Verification is skipped when unset.
NEXT_PUBLIC_RECAPTCHA_SITE_KEY	Client	Renders the reCAPTCHA v2 widget. The widget is omitted when unset.
POSTGRES_URL	Server	Postgres connection string. Feedback storage is disabled until this is set.
POSTGRES_USER	Server	Overrides any user embedded in POSTGRES_URL. Optional.
POSTGRES_PASSWORD	Server	Overrides any password embedded in POSTGRES_URL. Optional.
POSTGRES_SSL	Server	disable no-verify verify-full. Defaults by host: off for localhost, TLS without certificate validation otherwise.
POSTGRES_POOL_MAX	Server	Maximum pool connections per server instance. Defaults to 3.

Every server-side variable is optional in the sense that the code degrades rather than crashes when it is absent — but the assistant and durable feedback delivery are both inert until they are set.

9. External Service Dependencies

Service	Used by	Failure behaviour
OpenAI-compatible LLM provider	/api/chat	Returns 503 if unconfigured, 502 on upstream error; the rest of the site is unaffected
Google reCAPTCHA v2	/api/feedback	Verification skipped entirely if the secret is unset; returns 400 on a failed challenge
Postgres (feedback store)	/api/feedback	Returns 503 if unconfigured, 502 on a failed write; the local JSONL copy is written regardless but is not a backup of record (section 8.3)
Google Fonts (build time)	Root layout	Fonts are fetched and self-hosted at build time via next/font — no runtime dependency
Vercel Analytics	Root layout	Non-blocking; no effect on rendering or functionality

10. Architectural Observations

10.1 Strengths

- The build-time content model eliminates database, CMS and caching infrastructure for the policy content itself, and makes content consistency a compile-time guarantee rather than a runtime concern. The application's only stateful dependency is the Postgres store behind the feedback form.
- The static-first rendering profile means reading traffic — which will be the overwhelming majority — is served from CDN edge cache at negligible cost and near-zero latency, an appropriate profile for a national policy publication.
- Provider abstraction on the assistant endpoint means the LLM backend can be moved between OpenAI, Azure OpenAI and a sovereign self-hosted deployment by environment variable alone — a meaningful property for a government-facing artefact where data residency may become a requirement.
- Server-side input sanitisation, length caps and history trimming on the chat endpoint limit both prompt-injection surface and token cost exposure.

10.2 Points to address before publication

- Feedback submissions are now durably stored in Postgres rather than relying on the local .data/feedback.jsonl write, which remains an ephemeral, best-effort convenience on serverless hosting. What is still open is operational: provisioning POSTGRES_URL on the hosting platform, and deciding a backup/retention policy for the feedback_entries table.
- The chat endpoint has no rate limiting or abuse throttling. An unauthenticated public endpoint proxying a metered LLM provider is a direct cost-exposure path; per-IP throttling is advisable before the robots restrictions are lifted.
- Both indexing controls (robots.txt and the layout robots metadata) must be reversed at launch, and they live in different files — a single change will not suffice.

- There is no committed CI workflow. Adding a pipeline that runs the type check and lint on pull requests would protect the build, given that content corrections are code changes.

11. On-Premises Deployment Requirements for Public Launch

The items below follow from what the code requires to run and be durable, not from an existing deployment target — none of it is provisioned in the repository today. Section 8 describes the current (Vercel-shaped) hosting evidence; this section covers what a self-hosted deployment needs to add before it serves the general public.

11.1 Application host

- A persistent Node.js 20+ process (next start) behind a process supervisor (systemd, pm2, or a container orchestrator) that restarts the app on crash and on host reboot; running next start directly in a terminal is not resilient on its own.
- A reverse proxy (nginx, Caddy, HAProxy, etc.) in front of the Node process to terminate TLS and forward to the application's HTTP port — the application does not terminate TLS or serve HTTPS itself.
- If more than one application instance runs behind a load balancer, `POSTGRES_POOL_MAX` (section 8.6) is a per-instance setting, not a cluster-wide one; size it so instances × pool max stays within Postgres' own `max_connections`.

11.2 Postgres provisioning

- `POSTGRES_URL` must point at a reachable Postgres instance — self-hosted on the same network or a managed service. The database and role must already exist: the application only creates its own table (`CREATE TABLE IF NOT EXISTS` in `src/lib/db.ts`), not the database itself.
- The Postgres instance should not be publicly reachable: bind it to a private network and restrict inbound connections to the application host(s) only.
- TLS to Postgres: `POSTGRES_SSL=disable` (section 8.6) is appropriate only when the database sits on the same private network as the application. For anything reachable over a shared or public network, `verify-full` with a trusted certificate chain is the appropriate setting for a government-facing feedback channel — not the `no-verify` default that section 8.6 documents as convenient for managed providers.
- No backup or retention policy exists in the code. Section 10.2 already notes that `feedback_entries` is now the only durable store for public feedback; a public launch needs an operational backup schedule (e.g. `pg_dump` on a cron, or the platform's own snapshotting) before that store is relied on.

11.3 Secrets and configuration

- No `.env` file is committed (section 8.6); on-prem this means every server-side variable must be provisioned through the host's own secret mechanism (a systemd `EnvironmentFile`, a container secret store, etc.), not a committed file.

- Bot protection on `/api/feedback` is skipped entirely when `RECAPTCHA_SECRET_KEY` is unset (section 6.2). Both `RECAPTCHA_SECRET_KEY` and `NEXT_PUBLIC_RECAPTCHA_SITE_KEY` need to be set before the form is exposed to the public, or the endpoint accepts unverified submissions.

11.4 Vercel-specific code to remove or replace

- `@vercel/analytics` (section 2.1) reports to Vercel's collector and does nothing off that platform; it still ships to every client regardless of hosting target and should be removed or replaced with an on-prem-appropriate analytics approach before launch.

11.5 Hardening carried over from section 10.2

- The chat endpoint has no rate limiting; this is a direct cost-exposure risk once the site is public, since it proxies a metered LLM provider behind no authentication.
- Both indexing controls (`public/robots.txt` and the root layout's robots metadata, section 8.4) must be reversed at launch — leaving either one in place keeps the site out of search even once it is publicly reachable.
- There is no committed CI workflow; running the type check and lint on every change is advisable before the build is relied on for public content corrections.

11.6 Domain, TLS and DNS

- None of this is present in the repository: a certificate for the reverse proxy (an internal CA if the deployment stays intranet-only, or a public CA such as Let's Encrypt if it is internet-facing) and a DNS record pointing at the host or load balancer both need to be provisioned outside the codebase.

Prepared from the main branch on 27 July 2026, and revised on 11 August 2026